

Algorithmie – Algorithmes de Recherche

Table des matières

1 - Consolidation : parcours de tableau simple	2
Rappel rapide	2
Importance de l'index.....	3
2 - Recherche séquentielle (linéaire).....	4
Principe : parcourir élément par élément	4
Algorithme détaillé	4
Cas où l'élément existe.....	5
Cas où l'élément n'existe pas	5
Compteur de comparaisons (introduction à la notion de coût).....	6
Variante : retourner toutes les occurrences.....	7
3 - Notion de tableau trié	8
Qu'est-ce qu'un tableau trié ?	8
Pourquoi trier ?	9
Visualisation croissant/décroissant	9
Vérifier si un tableau est trié	10
4 - Recherche dichotomique (binaire).....	11
Principe : diviser pour régner	11
Analogie avec le dictionnaire	11
Algorithme détaillé avec indices (début, fin, milieu).....	12
Exemple pas à pas détaillé	13
Exemple : élément non trouvé	14
Comparaison avec recherche séquentielle.....	16
5 - Exercices pratiques.....	17
Exercice 1 : Recherche séquentielle simple.....	17
Exercice 2 : Compter les occurrences.....	17
Exercice 3 : Recherche dichotomique guidée.....	17

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☑ Jérôme CHRETIENNE
 ☑ Sophie POULAKOS
 ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

1- Consolidation : parcours de tableau simple

Rappel rapide

Avant d'aborder les algorithmes de recherche, rappelons les bases du parcours de tableau.

Tableau simple :

```
nombres ← [15, 8, 23, 42, 16, 4, 31]
```

Parcours complet :

```
POUR i de 0 à longueur(nombres) - 1  
    AFFICHER nombres[i]  
FIN POUR
```

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

Recherche d'un élément spécifique (version basique) :

```
elementRecherche ← 23
trouve ← faux

POUR i de 0 à longueur(nombres) - 1
    SI nombres[i] = elementRecherche
        trouve ← vrai
        AFFICHER "Élément trouvé à l'index " + i
    FIN SI
FIN POUR

SI trouve = faux
    AFFICHER "Élément non trouvé"
FIN SI
```

Importance de l'index

L'**index** est la position d'un élément dans le tableau.

C'est crucial pour :

- Savoir **où** se trouve un élément
- Pouvoir **modifier** un élément
- Pouvoir **supprimer** un élément

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

2- Recherche séquentielle (linéaire)

Principe : parcourir élément par élément

La **recherche séquentielle** (ou **linéaire**) est la méthode la plus simple :

- On examine chaque élément un par un
- De la première position jusqu'à la dernière
- On s'arrête quand on trouve l'élément (ou qu'on arrive à la fin)

Analogie : chercher un livre dans une bibliothèque non triée

- On regarde le premier livre
- Puis le deuxième
- Puis le troisième
- Etc. jusqu'à trouver le bon ou avoir tout parcouru

Algorithme détaillé

```

FONCTION rechercheSequentielle(tableau, element)
    POUR i de 0 à longueur(tableau) - 1
        SI tableau[i] = element
            RETOURNER i // Retourne la position
    FIN SI
FIN POUR
RETOURNER -1 // Élément non trouvé (convention : -1)
FIN FONCTION
```

Explication :

- On parcourt le tableau avec un compteur `i`
- À chaque tour, on compare `tableau[i]` avec l'élément recherché
- Si on trouve : on retourne l'index
- Si on arrive à la fin sans trouver : on retourne -1

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

Cas où l'élément existe

Exemple :

```
nombres ← [15, 8, 23, 42, 16]  
position ← rechercheSequentielle(nombres, 23)
```

Étapes :

```
i=0 : nombres[0] = 15 ≠ 23  
i=1 : nombres[1] = 8  ≠ 23  
i=2 : nombres[2] = 23 = 23 → TROUVÉ ! Retourne 2
```

Résultat : `position = 2`

Cas où l'élément n'existe pas

Exemple :

```
nombres ← [15, 8, 23, 42, 16]  
position ← rechercheSequentielle(nombres, 100)
```

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

Étapes :

```
i=0 : nombres[0] = 15  ≠ 100  
i=1 : nombres[1] = 8   ≠ 100  
i=2 : nombres[2] = 23  ≠ 100  
i=3 : nombres[3] = 42  ≠ 100  
i=4 : nombres[4] = 16  ≠ 100  
Fin du tableau → Retourne -1
```

Résultat : `position = -1` (non trouvé)

Compteur de comparaisons (introduction à la notion de coût)

Pour comprendre l'efficacité de l'algorithme, on peut compter le nombre de comparaisons.

```
FONCTION rechercheSequentielleAvecCompteur(tableau, element)  
    comparaisons ← 0  
  
    POUR i de 0 à longueur(tableau) - 1  
        comparaisons ← comparaisons + 1  
        SI tableau[i] = element  
            AFFICHER "Nombre de comparaisons : " + comparaisons  
            RETOURNER i  
        FIN SI  
    FIN POUR  
  
    AFFICHER "Nombre de comparaisons : " + comparaisons  
    RETOURNER -1  
FIN FONCTION
```

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☑ Jérôme CHRETIENNE
☑ Sophie POULAKOS
☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

Observations :

- **Meilleur cas** : l'élément est au début → 1 comparaison
- **Pire cas** : l'élément est à la fin (ou absent) → N comparaisons (N = taille du tableau)
- **Cas moyen** : environ N/2 comparaisons

Variante : retourner toutes les occurrences

Si un élément peut apparaître plusieurs fois :

```
FONCTION rechercherToutesOccurrences(tableau, element)
    positions ← [] // Tableau vide pour stocker les positions

    POUR i de 0 à longueur(tableau) - 1
        SI tableau[i] = element
            ajouter i dans positions
        FIN SI
    FIN POUR

    RETOURNER positions
FIN FONCTION
```

Exemple :

```
nombres ← [5, 3, 8, 3, 9, 3]
positions ← rechercherToutesOccurrences(nombres, 3)
→ positions = [1, 3, 5]
```

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

3- Notion de tableau trié

Qu'est-ce qu'un tableau trié ?

Un **tableau trié** est un tableau dont les éléments sont rangés dans un ordre précis :

- **Ordre croissant** : du plus petit au plus grand
- **Ordre décroissant** : du plus grand au plus petit

Exemples :

Non trié :

[15, 8, 23, 4, 16, 42]

Trié (croissant) :

[4, 8, 15, 16, 23, 42]

Trié (décroissant) :

[42, 23, 16, 15, 8, 4]

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☒ Jérôme CHRETIENNE
- ☒ Sophie POULAKOS
- ☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

Pourquoi trier ?

Trier un tableau présente plusieurs avantages :

1. Recherche plus rapide : algorithmes optimisés (recherche dichotomique)
2. Lisibilité : plus facile de comprendre les données
3. Trouver min/max : premier et dernier élément
4. Détecter les doublons : éléments identiques sont côte à côte
5. Organisation : notes par ordre de mérite, prix croissants, etc.

Visualisation croissant/décroissant

Ordre croissant :

[1, 2, 3, 4, 5, 6, 7, 8, 9]
↗ ↗ ↗ ↗ ↗ ↗ ↗ ↗

Ordre décroissant :

[9, 8, 7, 6, 5, 4, 3, 2, 1]
↘ ↘ ↘ ↘ ↘ ↘ ↘ ↘

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

Vérifier si un tableau est trié

```
FONCTION estTrieCroissant(tableau)
  POUR i de 0 à longueur(tableau) - 2
    SI tableau[i] > tableau[i + 1]
      RETOURNER faux
    FIN SI
  FIN POUR

  RETOURNER vrai
FIN FONCTION
```

Explication :

- On compare chaque élément avec le suivant
- Si un élément est plus grand que le suivant → pas trié croissant
- Si on arrive à la fin sans problème → trié croissant

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

4- Recherche dichotomique (binaire)

Principe : diviser pour régner

La **recherche dichotomique** est un algorithme beaucoup plus efficace que la recherche séquentielle.

Condition obligatoire : le tableau **DOIT** être trié.

Principe :

1. On regarde l'élément du milieu
2. Si c'est celui qu'on cherche → trouvé !
3. Sinon :
 - Si l'élément cherché est plus petit → on cherche dans la moitié gauche
 - Si l'élément cherché est plus grand → on cherche dans la moitié droite
4. On répète jusqu'à trouver ou ne plus avoir d'éléments

Analogie avec le dictionnaire

Chercher un mot dans un dictionnaire :

- On n'ouvre PAS le dictionnaire à la première page
- On l'ouvre **au milieu**
- Si le mot cherché est avant → on cherche dans la première moitié
- Si le mot cherché est après → on cherche dans la deuxième moitié
- On répète jusqu'à trouver le mot

C'est exactement le principe de la recherche dichotomique !

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

Algorithme détaillé avec indices (début, fin, milieu)

```
FONCTION rechercheDichotomique(tableau, element)
    debut ← 0
    fin ← longueur(tableau) - 1

    TANT QUE debut ≤ fin
        milieu ← (debut + fin) / 2 // Division entière
        SI tableau[milieu] = element
            RETOURNER milieu // Trouvé !
        SINON SI tableau[milieu] < element
            debut ← milieu + 1 // Chercher dans la moitié droite
        SINON
            fin ← milieu - 1 // Chercher dans la moitié gauche
        FIN SI
    FIN TANT QUE

    RETOURNER -1 // Non trouvé
FIN FONCTION
```

Variables importantes :

- `debut` : index de début de la zone de recherche
- `fin` : index de fin de la zone de recherche
- `milieu` : index de l'élément du milieu

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

Exemple pas à pas détaillé

Cherchons **34** dans ce tableau trié :

```
[4, 8, 15, 16, 23, 34, 42, 56, 67]
0  1  2  3  4  5  6  7  8  (indices)
```

Étape 1 :

```
debut = 0, fin = 8
milieu = (0 + 8) / 2 = 4
tableau[4] = 23
23 < 34 → chercher à droite
debut = 5
```

Étape 2 :

```
debut = 5, fin = 8
milieu = (5 + 8) / 2 = 6
tableau[6] = 42
42 > 34 → chercher à gauche
fin = 5
```

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

Étape 3 :

```
debut = 5, fin = 5  
milieu = (5 + 5) / 2 = 5  
tableau[5] = 34  
34 = 34 → TROUVÉ !  
RETOURNER 5
```

Résultat : élément trouvé à l'index 5 en **3 comparaisons** seulement !

Exemple : élément non trouvé

Cherchons **30** dans le même tableau :

[4, 8, 15, 16, 23, 34, 42, 56, 67]

Étape 1 :

```
debut = 0, fin = 8  
milieu = 4  
tableau[4] = 23  
23 < 30 → debut = 5
```

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

Étape 2 :

```
debut = 5, fin = 8  
milieu = 6  
tableau[6] = 42  
42 > 30 → fin = 5
```

Étape 3 :

```
debut = 5, fin = 5  
milieu = 5  
tableau[5] = 34  
34 > 30 → fin = 4
```

Étape 4 :

```
debut = 5, fin = 4  
debut > fin → arrêt de la boucle  
RETOURNER -1 (non trouvé)
```

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

Comparaison avec recherche séquentielle

Pour un tableau de 9 éléments :

Algorithme	Meilleur cas	Pire cas
Séquentielle	1 comparaison	9 comparaisons
Dichotomique	1 comparaison	4 comparaisons

Pour un tableau de 1000 éléments :

Algorithme	Pire cas
Séquentielle	1000 comparaisons
Dichotomique	10 comparaisons environ

Gain énorme avec la recherche dichotomique !

Mais **rappel** : le tableau DOIT être trié.

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Recherche

5- Exercices pratiques

Exercice 1 : Recherche séquentielle simple

Écrire un algorithme qui :

- Crée un tableau de prénoms
- Demande un prénom à rechercher
- Utilise la recherche séquentielle pour le trouver
- Affiche sa position ou "non trouvé"

Exercice 2 : Compter les occurrences

Écrire un algorithme qui :

- Crée un tableau de nombres (avec des doublons possibles)
- Demande un nombre à rechercher
- Compte combien de fois ce nombre apparaît
- Affiche le résultat

Exercice 3 : Recherche dichotomique guidée

Soit le tableau trié : `[2, 5, 8, 12, 16, 23, 38, 45, 56, 67, 78]`

1. Écrire l'algorithme de recherche dichotomique complet
2. Tracer l'exécution pour rechercher le nombre 23
3. Indiquer les valeurs de `debut`, `fin` et `milieu` à chaque étape

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.